وثيقة معمارية تطبيق ذاكرة

Architecture Documentation (arc42) — Dhakira

الإصدار: 1.0 · **الحالة:** معتمدة · **صاحب المنتج:** محمد

مبنية على وثيقة المتطلبات الإصدار 1.4، ونموذج C4 الإصدار 1.3.

عن هذه الوثيقة

تجمع هذه الوثيقة معمارية "ذاكرة" في مكان واحد: ما الذي يجب أن يحققه التطبيق، وما الذي يقيّده، وكيف بُني، ولماذا اختيرت كل طريقة دون غيرها. كُتبت لمن يريد أن يفهم التطبيق من الداخل دون أن يقرأ كل وثائق المشروع، سواء كان مبرمجاً سيكتب الكود أو يراجعه، أو قارئاً يريد أن يعرف كيف يُبنى تطبيق كهذا.

تتبع الوثيقة قالب arc42، وهو قالب معروف لتوثيق المعمارية من اثني عشر قسماً، يجيب كل قسم عن سؤال واحد. بعض الأقسام يلخص وثيقة منشورة أخرى ويحيل إليها، وبعضها مكتوب هنا لأول مرة: استراتيجية الحل، والنشر، والمفاهيم المشتركة، والمخاطر.

#	القسم	السؤال الذي يجيب عنه
1	الأهداف والمتطلبات	ماذا يجب أن يحقق التطبيق، ولمن؟
2	القيود	ما الذي لا نختاره، بل نعمل داخله؟
3	السياق والنطاق	أين تنتهي حدود التطبيق، ومع من يتعامل؟
4	استراتيجية الحل	ما الاختيارات الكبرى التي يقوم عليها كل شيء؟
5	وحدات البناء	من أي أجزاء يتكون التطبيق، وأين يُكتب كل نوع من الكود؟
6	سلوك التشغيل	ماذا يحدث، وبأي ترتيب، في المواقف المهمة؟
7	النشر	ماذا يوضع على جهاز المستخدم، وأين تُحفظ بياناته؟
8	المفاهيم المشتركة	ما القواعد التي تحكم التطبيق كله لا جزءاً منه؟
9	القرارات المعمارية	ما القرارات المهمة، ولماذا، وما الذي رُفض؟
10	الجودة	كيف نعرف أن أهداف الجودة تحققت؟
11	المخاطر والدين التقني	ما الذي نعرف أنه ناقص أو قد يسوء؟
12	المصطلحات	ما معنى كل مصطلح تستعمله الوثيقة؟

تصف الوثيقة الإصدار الأول كما صُمّم. وبعض ما فيها، مثل أرقام الأداء، لا يُحدد إلا بعد أول نسخة تعمل، وهو مذكور بصراحة في القسم 11. وإذا مرّ بك مصطلح غير مألوف، فمعناه في القسم 12.

الوثائق التي تحيل إليها هذه الوثيقة:

الوثيقة	ما تجده فيها
رؤية المنتج	لمن نبني التطبيق، والمشكلة التي يحلها، ومعيار النجاح.
وثيقة المتطلبات	كل ما يجب أن يفعله الإصدار الأول وما لا يفعله، برموز ثابتة.
النموذج المفاهيمي	المفاهيم التي يتعامل معها التطبيق والعلاقات بينها.
نموذج C4	مخططات السياق والحاويات ومكوّنات الخادم.
مخطط قاعدة البيانات	الجداول وحقولها وقواعد الحذف.
مخطط التسلسل لاستئناف المشروع	ما يحدث خطوة خطوة عند فتح مشروع.
عقد الواجهة البرمجية	كل طلب بين الواجهة والخادم: مدخلاته وردوده وأخطاؤه.

1. الأهداف والمتطلبات

يعرض هذا القسم باختصار ما يجب أن يحققه التطبيق، لأن كل قرار في الأقسام التالية يُقاس عليه. والتفاصيل الكاملة في رؤية المنتج ووثيقة المتطلبات.

1.1 ما يفعله التطبيق

"ذاكرة" تطبيق سطح مكتب لمطور يعمل على عدة مشاريع. يجمع ما يعرفه المطور عن مشاريعه مع ما يكتشفه التطبيق بنفسه من الجهاز، ويتذكر كيف تتغير حالة كل مشروع مع الوقت، ويُظهر الاختلاف حين لا يطابق ما كُتب ما وُجد. فإذا عاد المطور إلى مشروع تركه منذ شهور، عرف من شاشة واحدة أين توقف، وما الذي تغيّر، وما الذي يحتاج انتباهه.

وظائفه الرئيسية ست:

1. **بناء الذاكرة:** اكتشاف المشاريع من مجلدات المطور، وتعريف أجزائها، وإعادة ربطها إذا انتقلت.
2. **العودة إلى المشروع:** شاشة واحدة تقول أين توقف المطور، وما الذي تغيّر منذ زيارته، وما الذي يحتاج انتباهه.
3. **فهم الحالة:** فحص Git وبيئة التشغيل وDocker وإعدادات النشر، وتذكّر تطورها، وكشف الاختلافات.
4. **حفظ المعرفة:** دليل التشغيل، والوثائق، والخدمات المرتبطة، وسجل النشر.
5. **العملاء:** ربط المشروع بعميل، وسجل التسليمات.
6. **حماية الذاكرة:** نسخة احتياطية مشفّرة.

والفكرة التي يقوم عليها التصميم كله أن التطبيق مفيد حتى لو لم يكتب المطور عن مشروعه شيئاً، لأن الفحص يبني الذاكرة وحده، وما يكتبه المطور يضيف معنى فوقها.

1.2 أهداف الجودة

هذه أهم خمسة أهداف للجودة بترتيب أهميتها، وتفصيلها بأمثلة في القسم 10.

#	الهدف	ما يعنيه للمطور
1	الأمان	يفتح التطبيق على أي مشروع دون خوف: لا يشغّل كوده، ولا يغيّر شيئاً فيه أو في الجهاز، ولا يطلب أي كلمة سر.
2	الصدق	ما لم يُفحص لا يظهر سليماً أبداً، وكل معلومة تظهر مع مصدرها وعمرها.
3	السرعة الظاهرة	كل شاشة تظهر فوراً بآخر ما يعرفه التطبيق، والفحص يجري في الخلفية.
4	الموثوقية	فشل جزء أو مصدر لا يوقف الباقي، وكل عطل يظهر مع سببه، ولا يبقى فحص معلّقاً.
5	قابلية التطوير	دعم تقنية مشاريع جديدة، أو نظام تشغيل آخر، أو نسخة عبر الإنترنت لاحقاً، دون إعادة بناء التطبيق.

1.3 أصحاب المصلحة

من	ما يهمه	أين يجده في هذه الوثيقة
المطور مستخدم التطبيق	تطبيق آمن وسريع وصادق	الأقسام 1 و8 و10
صاحب المنتج	قرارات واضحة يمكن مراجعتها، ومخاطر معروفة	الأقسام 4 و9 و11
من يكتب الكود أو يراجعه	أين يُكتب كل نوع من الكود، وما القواعد التي لا تُكسر	الأقسام 5 و6 و8
قارئ من خارج المشروع، مثل عميل أو صاحب عمل	كيف بُني التطبيق، ولماذا بهذه الطريقة	الأقسام 4 و7 و9

2. القيود

القيد شرط لا نختاره في كل مرة، بل نعمل داخله. بعض هذه القيود من حدود الإصدار الأول، وبعضها من حجم الفريق، وبعضها اصطلاحات اتفقنا عليها قبل كتابة سطر من الكود.

2.1 قيود تقنية

#	القيد	أثره على المعمارية	الرمز في المتطلبات
1	يعمل الإصدار الأول على Windows فقط.	كل ما يخص نظام التشغيل معزول في مكوّن واحد. وعلى نظام آخر يتوقف التطبيق برسالة واضحة بدل أن يعمل نصف عمل.	CON-01
2	يعمل دون إنترنت، ما عدا التحديث من المستودع البعيد حين يطلبه المطور.	لا خادم بعيد ولا خدمة سحابية. والاتصال الوحيد بالشبكة يجريه Git بصلاحياته الموجودة على الجهاز.	CON-02
3	لا يعتمد على أي خدمة من الخدمات التي يفحصها.	قاعدة البيانات ملف لا يحتاج تثبيتاً، ولا يحتاج التطبيق أي خادم أو خدمة أخرى.	CON-03
4	كل بيانات التطبيق في مجلد واحد معروف.	قاعدة البيانات والملفات المرفوعة والملفات المؤقتة في مجلد بيانات واحد، والخادم لا يكتب خارجه.	CON-04
5	كل جزء من المشروع مستودع Git.	الفحص يبدأ دائماً من المستودع، والمجلد الذي ليس فيه Git يُضاف رابطَ وثيقة.	CON-05
6	لا يشغّل التطبيق كود أي مشروع، ولا يغيّر شيئاً في المشاريع أو في الجهاز.	الوصول إلى الجهاز كله يمر بطبقة واحدة تقرأ فقط، ولكل عملية فيها مهلة.	SAF-01, SAF-02
7	لا يطلب كلمات سر ولا يحفظها.	تُحفظ معلومات الوصول غير السرية فقط، وقيم ملف المتغيرات ممنوعة إلا لقائمة صغيرة معروفة بأنها غير حساسة.	SAF-04, SEC-01

2.2 قيود تنظيمية

#	القيد	أثره على المعمارية
1	فريق صغير جداً، ومستخدم واحد في الإصدار الأول.	برنامج خادم واحد، وقاعدة بيانات واحدة، وتشغيل واحد، بلا كلفة توزيع الخدمات.
2	لا حسابات ولا تسجيل دخول في الإصدار الأول، والتصميم لا يمنع نسخة عبر الإنترنت أو العمل كفريق لاحقاً.	عنوان الخادم وطريقة الدخول في مكان واحد في الواجهة، وكل قراءة في الخادم تمر بمكان واحد لكل وحدة، يُضاف فيه شرط المالك لاحقاً.
3	التعلم هدف معلن للمشروع.	عند الاختيار بين حل أسرع وحل أنظف نتعلم منه، نميل إلى الأنظف. ومع ذلك تُرفض الأنماط المصممة لفرق كبيرة، لأنها لا تفيد مستخدماً واحداً.
4	الخادم والواجهة يُكتبان بالتوازي.	الواجهة تُبنى أولاً على بيانات وهمية بشكل العقد بين الطرفين، ثم تنتقل إلى الخادم الحقيقي وحدةً بعد وحدة.

2.3 اصطلاحات

- الوثائق بالعربية، داخل مستودع الكود نفسه، والمخططات بـ Mermaid ليرسمها GitHub مباشرة، ولكل وثيقة منشورة نسخة PDF.
- كل قرار معماري مهم مكتوب بسببه وبالبديل الذي رُفض.
- العقد بين الواجهة والخادم موثق قبل الكود في عقد الواجهة البرمجية، والطرفان يلتزمان به.
- كل قاعدة على البيانات تُفرض في الخادم، حتى لو أخفت الواجهة الزر الذي سيُرفض.

3. السياق والنطاق

يرسم هذا القسم حدود التطبيق: ما بداخله، ومع من يتعامل من خارجه. والحد الأهم أن التطبيق يقرأ من الجهاز ولا يغيّر فيه شيئاً، ولا يتصل بالشبكة إلا عندما يطلب المطور التحديث من المستودع البعيد. مخطط السياق الكامل في نموذج C4، القسم 2.

3.1 السياق من جهة العمل

#	الطرف	ما يأخذه التطبيق منه	ما يعطيه التطبيق له
1	المطور	قراراته في الاكتشاف والاختلافات، وما يكتبه: نقطة التوقف، والقيم المتوقعة، والوثائق، والعملاء	شاشة الاستئناف، وحالة كل مشروع، والمشاكل مع أسبابها والخطوة المقترحة لكل منها
2	ملفات المشاريع	ملفات الإعدادات والمتطلبات، وأسماء المتغيرات في ملف المتغيرات، وملفات السجلات التي سجّلها المطور	لا شيء، فالتطبيق يقرأ فقط
3	Git	حالة كل مستودع: الفرع، والملفات المعدّلة، وما لم يُرفع وما لم يُسحب، وآخر commits	لا شيء، ما عدا طلب التحديث من المستودع البعيد حين يطلبه المطور
4	المستودع البعيد	آخر حالة للفروع البعيدة، عبر Git فقط	لا شيء
5	Windows	الخدمات المثبتة، والمنافذ المستعملة، والبرامج قيد التشغيل	لا شيء
6	Docker	الحاويات وحالتها وسجلاتها، وغيابه ليس خطأ	لا شيء
7	أدوات المطور	وجودها على الجهاز فقط	يفتحها على مسار المشروع، ولا يشغّل بها شيئاً

3.2 السياق التقني

كل القنوات محلية على جهاز المطور، والاستثناء الوحيد التحديث من المستودع البعيد. ولكل قناة تقنية واحدة معروفة.

#	القناة	التقنية	ملاحظة
1	الواجهة والخادم	JSON/HTTP	على الجهاز نفسه فقط. لا يقبل الخادم طلباً من جهاز آخر ولا من صفحة ويب.
2	الخادم وملفات المشاريع	قراءة ملفات	داخل المجلدات التي سجلها المطور فقط.
3	الخادم وGit	Git CLI	أوامر قراءة ثابتة، ولكل أمر مهلة.
4	Git والمستودع البعيد	HTTPS, SSH	بصلاحيات Git الموجودة على الجهاز، والتطبيق لا يحفظها.
5	الخادم وWindows	Windows APIs	الخدمات والمنافذ والبرامج قيد التشغيل.
6	الخادم وDocker	Docker Engine API	قراءة فقط.
7	الخادم وأدوات المطور	تشغيل برنامج	فتح الأداة فقط، منفصلة عن التطبيق.

4. استراتيجية الحل

هنا خلاصة المعمارية: الشكل العام، والتقنيات، وطريقة تنظيم الكود في الطرفين. كل اختيار مكتوب مع سببه، والبدائل التي رُفضت في القسم 9.

4.1 الاختيارات الأساسية

#	السؤال	الاختيار	لماذا
1	ما شكل النظام؟	عميل وخادم على الجهاز نفسه (Server–Client): واجهة للعرض، وخادم يحمل المنطق والبيانات، ولا يتكلمان إلا عبر عقد موثق.	المنطق كله في مكان واحد، والخادم نفسه يمكن نشره لاحقاً دون إعادة كتابة الواجهة.
2	كيف يُبنى الخادم؟	برنامج واحد وقاعدة بيانات واحدة، مقسوم إلى وحدات لكل منها حدودها (Modular Monolith).	بساطة البرنامج الواحد لفريق صغير، مع حدود واضحة تسمح بفصل وحدة إذا احتجنا ذلك.
3	بأي تقنيات؟	الواجهة بـ Flutter لسطح مكتب Windows. والخادم بـ Django وDjango REST Framework، ويخدمه waitress. وقاعدة البيانات SQLite.	Django خبرة الفريق، وwaitress يعمل على Windows وعلى Linux عند النشر لاحقاً، وSQLite ملف واحد لا يحتاج تثبيتاً ويسمح بالقراءة أثناء الكتابة.
4	كيف يصل إلى المستخدم؟	تطبيق واحد مدمج: برنامج تثبيت واحد يضع الواجهة والخادم معاً، والواجهة تشغّل الخادم في الخلفية وتغلقه.	يفتح المستخدم التطبيق بأيقونة واحدة، ولا يثبّت أي شيء آخر.
5	كيف يصل إلى الجهاز؟	طبقة محلية واحدة هي الطريق الوحيد إلى الملفات وGit وDocker ونظام التشغيل، وما يخص Windows معزول داخلها.	ضمان القراءة فقط في مكان واحد، وهذه الطبقة يمكن فصلها لاحقاً برنامجاً مستقلاً.
6	كيف يجري الفحص؟	عند الطلب وفي الخلفية، بخيطي تنفيذ وكاتب واحد لقاعدة البيانات. والشاشة تظهر فوراً بآخر حالة معروفة مع عمرها.	لا ينتظر المطور فحصاً قد يطول، ولا تتزاحم الكتابة في قاعدة البيانات.
7	كيف يتذكر؟	ذاكرة تلقائية تحفظ لقطة حالة لكل جزء عندما تتغير فقط، وتحذف القديم بسياسة ثابتة.	تاريخ المشروع يتكوّن دون أن يكتب المطور شيئاً، ولا تمتلئ الذاكرة بلقطات متطابقة.
8	كيف يُنظَّم كود الخادم؟	في كل وحدة: استقبال الطلب، ثم خدمات تحمل المعنى والكتابة، وقراءات منفصلة. وفوق الوحدات طبقة للأفعال التي تكتب في أكثر من وحدة.	لكل نوع من الكود مكان واحد، ولا اعتماد دائري بين الوحدات.
9	كيف يُنظَّم كود الواجهة؟	نمط MVVM مع Riverpod: الشاشة ترسم، ومدير الشاشة يحمل حالتها، ووسيط البيانات يرسل الطلبات عبر نقطة اتصال واحدة بالخادم.	لا منطق عمل في الواجهة، وأي تغيير في العقد يُصلح في سطر واحد لا في الشاشات.

4.2 من التطبيق المحلي إلى الإنترنت

الإصدار الأول تطبيق محلي لمستخدم واحد. وإذا صار له مستخدمون، يُنشر الخادم على الإنترنت وتُضاف الحسابات. صُممت المعمارية بحيث يكون هذا الانتقال إضافة لا إعادة بناء.

القطعة	الآن	بعد النشر
الواجهة	تتصل بالخادم على الجهاز نفسه، بلا تسجيل دخول	تتصل بعنوان على الإنترنت، مع شاشة دخول
الخادم	على جهاز المطور، بقاعدة بيانات في ملف، والفحص في خيط داخله	على الإنترنت، مع الحسابات وقاعدة بيانات أكبر
الطبقة المحلية	جزء من الخادم	برنامج صغير على جهاز المستخدم، يسمى الوكيل المحلي، لأن الخادم البعيد لا يرى الجهاز

ثلاثة قرارات تُبقي هذا الطريق مفتوحاً. الأول أن العنوان وطريقة الدخول في مكان واحد في الواجهة. والثاني أن طلب الفحص يرجع رقمه فوراً ثم يُسأل عن حالته، وهو النمط نفسه الذي سيعمل به الوكيل المحلي. والثالث أن كل قراءة في الخادم تمر بمكان واحد لكل وحدة، فيُضاف فيه شرط "لهذا المستخدم فقط".

وأصعب ما في النشر معروف من الآن: كل معلومة ستحتاج مالكاً، والوكيل المحلي سيجيب عن كل طلب يقرأ الجهاز، لا عن الفحص وحده.

4.3 أنماط معروفة في التصميم

لم نخترع طرقاً جديدة. كل جزء من التصميم يستعمل نمطاً معروفاً في مكانه المناسب فقط:

- **Strategy لمكتشفات التقنية:** لكل تقنية مكتشف بالواجهة نفسها، وDjango أولها. ودعم تقنية جديدة يعني مكتشفاً جديداً بجانب الموجود.
- **Adapter لنظام التشغيل:** كل ما يخص Windows خلف واجهة واحدة يختارها الخادم مرة واحدة عند بدئه.
- **المنافذ والمحوّلات (Ports and Adapters) على أطراف الجهاز فقط:** الطبقة المحلية واجهات مجردة، خلفها نسخة حقيقية ونسخة وهمية للاختبار.
- **طبقة الخدمات وطبقة الأفعال (Service Layer, Application Services):** في الخادم، لفصل المنطق عن استقبال الطلبات.
- **جانب قراءة خاص بالشاشات (Backend for Frontend):** وحدة واحدة تجمع ما تعرضه كل شاشة، بالقراءة والحساب فقط.
- **MVVM مع وسيط البيانات (Repository):** في الواجهة، كما يوصي به دليل Flutter.

5. وحدات البناء

يفتح هذا القسم التطبيق طبقة بعد طبقة: من الحاويات، إلى وحدات الخادم، إلى طريقة تنظيم الكود في الطرفين. مخططات الحاويات والمكوّنات في نموذج C4، وهنا ما يربطها بالكود.

5.1 المستوى الأول: الحاويات

يرى المستخدم برنامجاً واحداً، لكنه من الداخل أربع حاويات بمصطلحات C4، أي أربعة أجزاء يعمل كل منها وحده أو يخزن بيانات. وهي ليست حاويات Docker.

#	الحاوية	التقنية	مسؤوليتها
1	Desktop App	Flutter, Windows	الواجهة بلغتين. تشغّل الخادم وتغلقه، وتعرض آخر حالة معروفة فوراً، وتتابع الفحص الجاري.
2	API Application	Python, Django + DRF, waitress	منطق التطبيق كله والفحص. وهو وحده يملك البيانات ويصل إلى الجهاز.
3	Database	SQLite	المعرفة، والفحوصات، وتاريخ لقطات الحالة، وسجل النشاط، في ملف واحد.
4	File Store	Windows folder	الملفات التي يرفعها المطور، بجانب قاعدة البيانات في مجلد البيانات.

5.2 المستوى الثاني: داخل الخادم

يقسم نموذج C4 الخادم إلى اثني عشر مكوّناً في أربع طبقات: الدخول، ومنطق التطبيق، والفحص، والوصول المحلي. المكوّن هناك دور، أما في الكود فالخادم تسع وحدات، والوحدة مجلد له جداوله وحدوده. لذلك قد يتوزع مكوّن واحد على أكثر من وحدة، وقد تحمل وحدة واحدة أكثر من مكوّن.

#	الوحدة	ما تملكه	المكوّن في C4
1	projects	المشاريع وأجزاؤها، والمجلدات الرئيسية والمرفوضة، واكتشاف المشاريع	Discovery & Projects
2	memory	الفحص كله: تنظيمه، واكتشاف تقنية كل جزء، وتقييم الفحوصات، وحفظ لقطات الحالة	Scan Coordinator, Stack Detectors, Check Engine, Automatic Memory
3	resume	ملاحظة "أين توقفت"، والخطوات التالية، وروابط أدوات المهام	Knowledge
4	expectations	القيم المتوقعة، وقرارات المطور في الاختلافات، والمشاكل المخفية، والاقتراحات المتجاهلة	Knowledge
5	docs	الوثائق وأماكنها، والملاحظات، ودليل التشغيل، وأماكن سجلات الأخطاء	Knowledge
6	services	الخدمات وحسابات المزوّدين، وبيئات المشروع، وسجل النشر	Knowledge
7	clients	العملاء وعلاقتهم بالمشاريع، والتسليمات	Knowledge
8	core	الإعدادات، وسجل النشاط، وأدوات المطور، والإغلاق، وشروط الأمان المشتركة	Knowledge, API Layer
9	screens	ما تعرضه كل شاشة، بالقراءة والحساب فقط، بلا جداول	Views

وبجانب الوحدات التسع ثلاثة أجزاء بلا جداول. طبقة الأفعال المشتركة فوق كل الوحدات، وفيها كل فعل من المستخدم يكتب في أكثر من وحدة، ومنها التصدير والاستيراد، أي المكوّن Backup في C4. والطبقة المحلية في الأسفل، وفيها Local Access Layer وAdapter Windows، ولا تعرف Django ولا قاعدة البيانات. وقائمة الفحوصات في الأسفل أيضاً: تعريف كل فحص وخطورته ونطاقه، لأن أكثر من وحدة تحتاجها. ونقطة بداية الخادم (Startup) تشغّل ذلك كله.

القواعد التي تحكم الخادم:

- كل وحدة تكتب في جداولها فقط، ولا تقرأ جداول وحدة أخرى إلا عبر قراءاتها المعلنة.
- الاعتماد بين الوحدات باتجاه واحد، بلا دوائر. والقراءة التي تعكس اتجاهاً موجوداً تنتقل إلى طبقة الأفعال المشتركة.
- الفعل الذي يكتب في أكثر من وحدة مكانه طبقة الأفعال المشتركة، وينفَّذ في عملية حفظ واحدة: كله أو لا شيء. وإذا احتاج فحصاً، يطلبه بعد الحفظ.
- استقبال الطلب يتحقق من شكله فقط، مثل وجود الحقل ونوعه. أما المعنى، مثل "هل الاسم مستعمل؟" و"هل المسار مستودع Git؟"، فمكانه خدمات الوحدة.
- القاعدة التي يجب ألا تُكسر أبداً تُكتب قيداً في قاعدة البيانات نفسها، لا في الكود وحده.
- كل ما يلمس الجهاز يمر بالطبقة المحلية. وللفحص مدخل واحد، يطلبه منه أي جزء يحتاج فحصاً، ولا يشغّل أحد خيطاً بنفسه.
- حفظ الفحوصات ولقطات الحالة له كاتب واحد هو الذاكرة التلقائية.
- لا أحداث مخفية: أثر كل فعل مكتوب صريحاً في المكان الذي يحدث فيه. والاستثناء الوحيد المشترك كتابة سطر في سجل النشاط، لأن كل الوحدات تسجّل أحداثها.

5.3 المستوى الثاني: داخل الواجهة

تتبع الواجهة نمط MVVM كما يوصي به دليل Flutter، مع مكتبة Riverpod لإدارة الحالة. والقاعدة الحاكمة: منطق العرض مسموح، ومنطق العمل ممنوع. فالواجهة تقرر أي نافذة تُفتح، وتترجم رمز الخطأ إلى جملة، وتُبرز ما تغيّر بعد الفحص. أما "هل الاسم مستعمل؟" وحساب حالة الأقسام والمقارنات، فمكانها الخادم.

#	الطبقة	دورها	ما لا تفعله
1	الشاشة	ترسم، وتنقل ضغطات المستخدم إلى مديرها	لا تعرف الخادم ولا أي عنوان
2	مدير الشاشة	يحمل حالة الشاشة (تحميل، أو بيانات، أو خطأ) وأفعالها، ويعيد القراءة بعد الكتابة أو الفحص	لا يحمل منطق عمل
3	وسيط البيانات	دالة لكل طلب في العقد، ووسيط لكل وحدة في الخادم	لا يعرف أي شاشة، ولا يعرف وسيطاً آخر
4	نقطة الاتصال الوحيدة	تعرف عنوان الخادم والرمز السري، وتحوّل كل رد خطأ إلى شكل واحد، وتعالج حالة "الخادم لا يرد"	لا تعرف أي شاشة

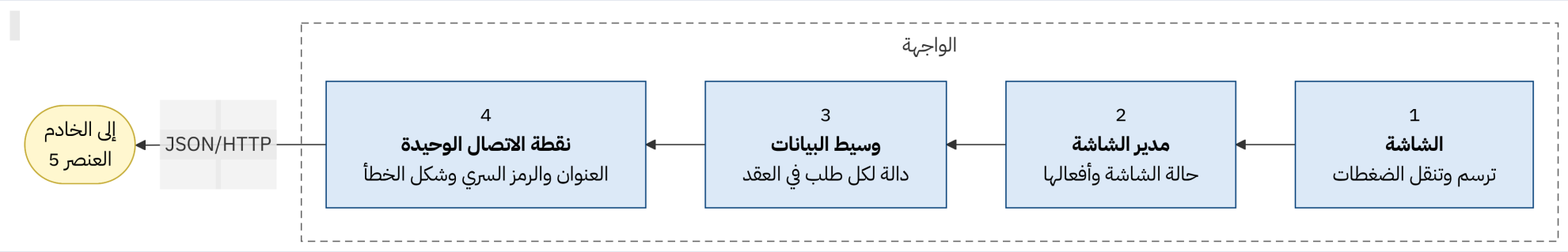
لكل شاشة مجلد يحمل اسمها كما يظهر في الشريط الجانبي بالواجهة الإنجليزية، فيجد المبرمج أي شاشة من اسمها. وأقسام صفحة المشروع مجلدات متجاورة، لا تُجمع حسب مجموعات الشريط، حتى لا يكسر نقل قسم من مجموعة إلى أخرى شيئاً. ومجلدات الشاشات لا تعتمد على بعضها إلا عبر مجلد مشترك.

ولكل وسيط بيانات ثلاث نسخ تُبدَّل بسطر واحد، والشاشات لا تعرف أيها يعمل: نسخة حقيقية تتصل بالخادم، ونسخة تُولَّد من وصف الواجهة البرمجية إن توفر، ونسخة وهمية ترجع بيانات بشكل العقد. الوهمية تسمح ببناء الشاشات قبل جاهزية الخادم وتُستعمل في الاختبار، ولا تدخل نسخة الإطلاق أبداً.

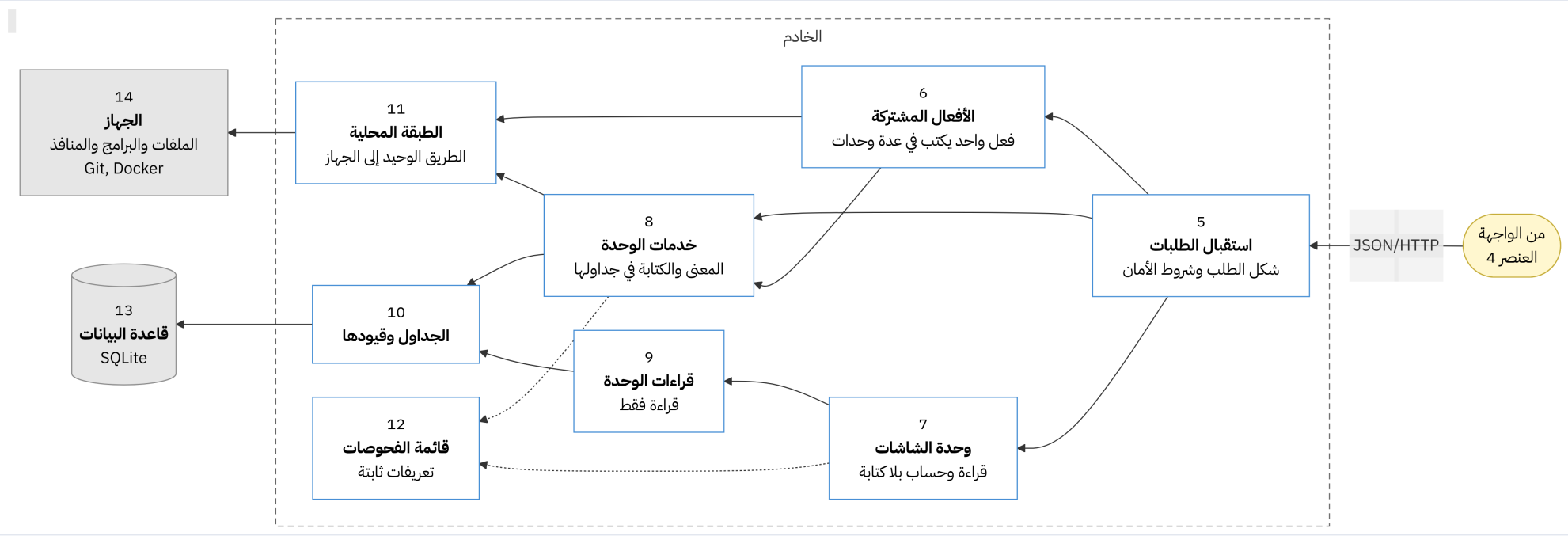
5.4 طبقات الكود في الطرفين

يجمع هذا المخطط ما سبق: الطلب يبدأ من الشاشة، ويمر بالطبقات من الأعلى إلى الأسفل، ولا يعود سهم إلى أعلى. والسهم المتقطع قراءة لتعريفات ثابتة.

الجزء أ: الواجهة



الجزء ب: الخادم



جدول توصيف المخطط

#	العنصر	الطرف	الوصف
1	الشاشة	الواجهة	ترسم البيانات، وتنقل ضغطات المستخدم إلى مديرها. لا تحتوي أي رابط أو طلب.
2	مدير الشاشة	الواجهة	يحمل حالة الشاشة وأفعالها، ويعيد القراءة بعد الكتابة أو انتهاء الفحص.
3	وسيط البيانات	الواجهة	دالة لكل طلب في العقد، ووسيط لكل وحدة في الخادم، بثلاث نسخ: حقيقية، ومولّدة، ووهمية.
4	نقطة الاتصال الوحيدة	الواجهة	المكان الوحيد الذي يعرف عنوان الخادم والرمز السري وشكل رد الخطأ.
5	استقبال الطلبات	الخادم	يطبّق شروط الأمان، ويتحقق من شكل الطلب، ثم يمرره إلى فعل مشترك أو خدمة أو وحدة الشاشات، ويرد.
6	الأفعال المشتركة	الخادم	فعل واحد من المستخدم يكتب في أكثر من وحدة، في عملية حفظ واحدة. يستدعي خدمات الوحدات، ويطلب الفحص بعد الحفظ إن لزم.
7	وحدة الشاشات	الخادم	تجمع ما تعرضه كل شاشة من قراءات الوحدات، وتحسب المقارنات وحالة الأقسام والترتيب، بلا كتابة وبلا قراءة من الجهاز.
8	خدمات الوحدة	الخادم	منطق العمل والمعنى، والكتابة في جداول الوحدة نفسها فقط. وفي وحدة الفحص: تنظيم الفحص وحفظ اللقطات.
9	قراءات الوحدة	الخادم	استعلامات القراءة لجداول الوحدة، وهي الطريق الوحيد لوحدة أخرى لتقرأ منها.
10	الجداول وقيودها	الخادم	الجداول، والقواعد التي يجب ألا تُكسر مكتوبة قيوداً في قاعدة البيانات.
11	الطبقة المحلية	الخادم	الطريق الوحيد إلى الجهاز، للقراءة وفتح الأدوات فقط. واجهات مجردة خلفها نسخة لـ Windows ونسخة وهمية للاختبار.
12	قائمة الفحوصات	الخادم	تعريف كل فحص: خطورته، ونطاقه، وطريقة المقارنة، والخطوة المقترحة.
13	قاعدة البيانات	التخزين	ملف SQLite واحد في مجلد البيانات.
14	الجهاز	خارج التطبيق	ملفات المشاريع، وGit، وDocker، وخدمات Windows ومنافذه وبرامجه.

6. سلوك التشغيل

المخططات الثابتة لا تقول متى يحدث كل شيء. يصف هذا القسم خمسة مواقف مهمة خطوة خطوة، وأولها فتح مشروع، وتسلسله الكامل في مخطط التسلسل لاستئناف المشروع.

6.1 فتح مشروع

1. يختار المطور مشروعاً، فترسل الواجهة طلبين معاً: طلب بيانات شاشة الاستئناف، وطلب فحص للمشروع بلا تحديث من المستودع البعيد.
2. يبني الخادم بيانات الشاشة من آخر لقطات محفوظة ومن معرفة المطور، فتظهر الشاشة فوراً مع عمر كل معلومة وإشارة "جارٍ الفحص".
3. يرجع طلب الفحص رقمه فوراً. وإذا كان المشروع يُفحص الآن، يرجع رقم ذلك الفحص بدل أن يبدأ فحصاً ثانياً. وإذا كان ينتظر دوره في فحص كل المشاريع، ينتقل إلى خيط المشروع المفتوح بالفحص نفسه.
4. يجمع الخادم حالة الجهاز مرة واحدة، ثم حالة كل جزء بالتوازي، ولكل جزء مهلة. ويُحفظ كل جزء فور انتهاء فحصه، بالدور.
5. تسأل الواجهة عن حالة الفحص كل ثانيتين تقريباً. وعند انتهائه تطلب بيانات الشاشة من جديد، فتتحدث الشاشة وتُبرز ما تغيّر منذ زيارة المطور السابقة.
6. الجزء الذي تعذر فحصه يظهر "لم يُفحص" مع السبب، وتبقى آخر لقطة له بعمرها الصادق.

ومغادرة شاشة المشروع لا تلغي الفحص: تتوقف المتابعة فقط، ويكمل الفحص ويحفظ ما وجده.

6.2 فحص كل المشاريع

يجري هذا الفحص في خيط الخلفية، لأنه أطول، وقد يشمل التحديث من المستودع البعيد. ويبدأ من جديد في كل مرة، ولا يكمل فحصاً سابقاً لم ينتهِ. ومعه يبحث التطبيق عن أدوات المطور المثبتة على الجهاز.

عند انتهاء كل مشروع، لا بعد انتهاء الطلب كله، يتحدث الشريط الجانبي والشاشة المفتوحة. وإذا فتح المطور مشروعاً لم يصله الدور بعد، يفحصه خيط المشروع المفتوح ويُشطب من قائمة الخلفية، فلا ينتظر المطور تحديثاً بطيئاً ليرى مشروعه.

6.3 بدء التطبيق

1. إذا كانت نسخة من التطبيق مفتوحة، تُظهر الواجهة النافذة الموجودة وتخرج.
2. ترسم الواجهة شاشة البدء فوراً بلغة المطور ومظهره وحجم النص، من نسخة صغيرة من الإعدادات تحفظها عندها. وعلى نظام غير Windows تعرض رسالة واضحة وتتوقف.
3. تشغّل الواجهة الخادم في الخلفية، وتمنحه رمزاً سرياً جديداً.
4. يتأكد الخادم أنه الوحيد الذي يعمل على مجلد البيانات، وإلا يرفض البدء برمز خطأ واضح. وتعيد شاشة البدء المحاولة بضع ثوانٍ قبل أن تعرض الخطأ.

5. يعالج الخادم ما بقي من تشغيل سابق. الفحص الذي كان جارياً يُسجَّل منقطعاً بسبب توقف مفاجئ، ويبقى ما اكتمل من أجزائه. والفحوص التي لم تبدأ تُلغى بصمت، والملفات المؤقتة تُحذف. وإذا انقطع فحص فعلاً، تظهر للمطور رسالة واحدة بذلك.
6. يختار الخادم منفذاً حراً ويبلغ الواجهة به، ثم يبدأ استقبال الطلبات وخيطي الفحص، ويبحث عن أدوات المطور.
7. تنتظر الواجهة أول جواب من الخادم، وتقارن إصداره بإصدارها، ثم تقرأ الإعدادات منه وتصحح نسختها إن اختلفت، وتفتح الشاشة الرئيسية.

6.4 الإغلاق والتوقف المفاجئ

لا يسأل التطبيق "فحص جارٍ، هل تغلق؟"، لأن الفحص لا يغيّر شيئاً، وفتح المشروع يفحصه من جديد. لكن الإغلاق يجب ألا يترك فحصاً معلّقاً ولا خادماً يعمل وحده.

الحالة	ما يحدث	ما يراه المطور بعدها
إغلاق عادي	يظهر تذكير "أين توقفت" الاختياري إن لزم. ثم تبلغ الواجهة الخادم بالإغلاق، فيرفض أي فحص جديد، ويلغي الفحوص المنتظرة، وينتظر انتهاء حفظ الجزء الجاري، ويسجّل الفحص الجاري منقطعاً بسبب الإغلاق. ثم توقف الواجهة الخادم بعد رده، أو بعد خمس ثوانٍ بلا رد.	لا رسالة. والمشروع الذي انقطع فحصه يظهر أن فحصه الأخير انقطع، وما اكتمل من أجزائه محفوظ.
إغلاق غير عادي للواجهة	مثل إنهائها من مدير المهام. يلاحظ الخادم غياب الواجهة، فيسلك طريق الإغلاق العادي نفسه ثم يخرج، فلا يبقى خادم يعمل وحده.	كما في الإغلاق العادي.
توقف الخادم فجأة	تلاحظ الواجهة ذلك، فتبلغ المطور وتعيد تشغيل الخادم. وعند بدئه يعالج الفحص المعلّق كما في الخطوة 5 من بدء التطبيق.	رسالة واحدة واضحة، ولا فحص معلّق.

6.5 فعل يكتب في أكثر من وحدة

يزيل المطور مشروعاً له عميل، ويختار إبقاء سجل العميل. ترسل الواجهة طلباً واحداً، وتنفذه طبقة الأفعال المشتركة في عملية حفظ واحدة: تفك ارتباط التسليمات والملاحظات والوثائق الخاصة بالعميل عن المشروع، فتبقى في صفحة العميل، ثم تحذف المشروع وما كتبه المطور عنه. وإذا فشلت أي خطوة، لا يُحفظ شيء. ولا يلمس التطبيق ملفات المشروع على الجهاز.

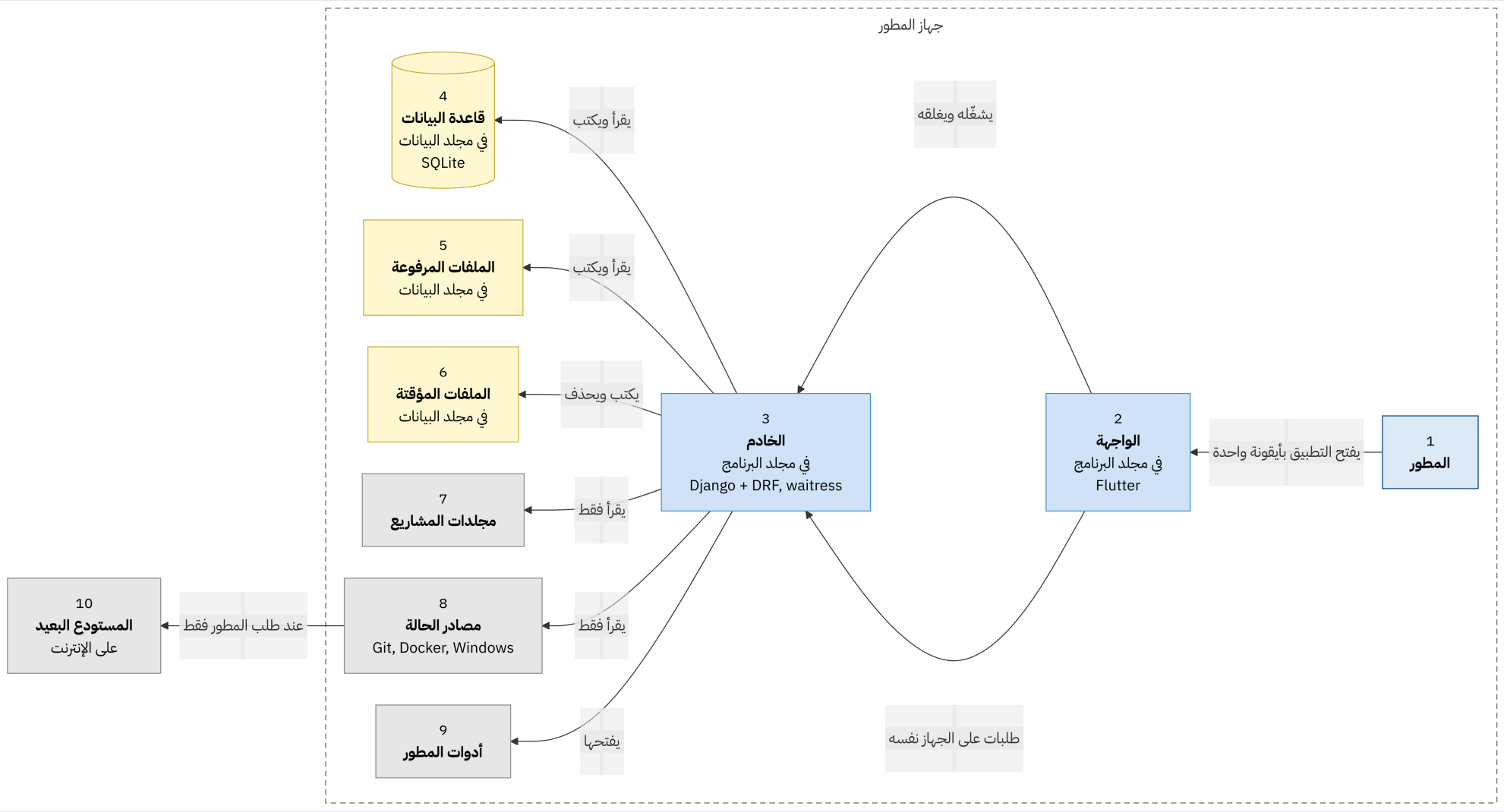
وبالطريقة نفسها يعمل تغيير تقنية جزء: يُحفظ اختيار المطور أولاً، ثم يُطلب فحص للمشروع بعد الحفظ، فلا يفحص التطبيق بيانات لم تُحفظ بعد.

7. النشر

يرى المستخدم برنامجاً واحداً يثبّته ويفتحه بأيقونة واحدة. يصف هذا القسم ما يوضع على جهازه فعلاً، وأين تُحفظ بياناته، وكيف تختلف بيئة التطوير عن النسخة التي تصل إليه.

7.1 مخطط النشر

كل ما داخل الإطار المتقطع على جهاز المطور. والصناديق الزرقاء ملفا التشغيل في مجلد البرنامج، والصفراء ما يكتبه التطبيق في مجلد البيانات، والرمادية ما يقرؤه أو يفتحه فقط.



جدول توصيف المخطط

#	العنصر	المكان	الوصف
1	المطور	مستخدم	المستخدم الوحيد، على جهاز Windows واحد.
2	الواجهة	مجلد البرنامج	ملف تشغيل Flutter. أول ما يعمل، وهي التي تشغّل الخادم وتغلقه.
3	الخادم	مجلد البرنامج	ملف تشغيل واحد يضم Python وDjango وكل ما يحتاجه الخادم، فلا يثبّت المستخدم أي شيء آخر.
4	قاعدة البيانات	مجلد بيانات المستخدم	ملف SQLite واحد: المعرفة، والفحوصات، واللقطات، وسجل النشاط.
5	الملفات المرفوعة	مجلد بيانات المستخدم	الوثائق المهمة التي رفعها المطور.
6	الملفات المؤقتة	مجلد بيانات المستخدم	ما يحتاجه بناء النسخة الاحتياطية واستيرادها، ويُحذف عند كل بدء للخادم.
7	مجلدات المشاريع	الجهاز	يقرؤها الخادم فقط، داخل المجلدات التي سجلها المطور.
8	مصادر الحالة	الجهاز	Git، وDocker إن كان يعمل، وخدمات Windows ومنافذه وبرامجه.
9	أدوات المطور	الجهاز	المحرر، ونافذة الأوامر، والمتصفح، وأداة قاعدة البيانات. يفتحها الخادم فقط.
10	المستودع البعيد	الإنترنت	الاتصال الوحيد بالشبكة، يجريه Git بصلاحياته عندما يطلب المطور التحديث.

7.2 التثبيت

برنامج تثبيت واحد يضع ملفي التشغيل، الواجهة والخادم، متجاورين في مجلد البرنامج. يُغلَّف الخادم بـ PyInstaller في ملف تشغيل واحد، ويُبنى برنامج التثبيت بـ Inno Setup. فلا يحتاج المستخدم Python ولا قاعدة بيانات ولا أي خدمة أخرى.

7.3 مجلد البيانات

كل بيانات التطبيق في مجلد واحد داخل بيانات المستخدم المحلية في Windows. اختير المجلد المحلي لا المتزامن بين الأجهزة، لأن مسارات المشاريع لا معنى لها على جهاز آخر، ولأن قاعدة البيانات قد تكبر. ولم يُختر مجلد البرنامج، لأن الكتابة فيه ممنوعة، ولأنه يُستبدل عند التحديث.

والخادم لا يكتب خارج هذا المجلد أبداً. حتى ملف النسخة الاحتياطية يرجعه إلى الواجهة، فتعرض نافذة الحفظ المعروفة في Windows، ويختار المطور مكانه.

ولا يعمل على المجلد إلا خادم واحد في الوقت نفسه. يحجزه الخادم عند بدئه، ويحرره Windows وحده إذا توقف الخادم، فلا يكتب خادمان في قاعدة بيانات واحدة.

7.4 التطوير ونسخة الإطلاق

النسختان من الكود نفسه، والفرق بينهما كله في مكان واحد في الواجهة. ويُحدَّد الوضع عند بناء التطبيق، فلا تخرج نسخة تطوير إلى المستخدم خطأً.

	التطوير	نسخة الإطلاق
من يشغّل الخادم	المطور، بخادم التطوير الذي يأتي مع Django	الواجهة، عند فتح التطبيق
المنفذ	ثابت من إعداد محلي	حر، يختاره الخادم في كل تشغيل
الرمز السري	ثابت من إعداد محلي	جديد في كل تشغيل
مجلد البيانات	مجلد خاص بالتطوير، منفصل عن الاستخدام اليومي	مجلد بيانات المستخدم
صفحة تجربة الطلبات	مفتوحة إن وُجدت	غير موجودة
البيانات الوهمية في الواجهة	مسموحة لبناء الشاشات واختبارها	ممنوعة

وعند كل بدء تقارن الواجهة إصدار الخادم بإصدارها. فإذا اختلفا، فالتثبيت ناقص، وتظهر رسالة واضحة بدل أعطال غامضة.

8. المفاهيم المشتركة

بعض القواعد لا تخص وحدة واحدة، بل تحكم التطبيق كله. جمعها هنا يمنع أن تُحل المشكلة نفسها بطريقتين في مكانين.

8.1 القراءة فقط

لا يشغّل التطبيق كود أي مشروع، ولا يغيّر شيئاً في المشاريع أو في الجهاز. ويُضمن ذلك في مكان واحد: الطبقة المحلية هي الطريق الوحيد إلى الجهاز، ولا تملك إلا عمليات قراءة وفتح أدوات. وإذا أمكن معرفة المعلومة بقراءة ملف، يقرؤها التطبيق من الملف بدل أن يشغّل أداة.

ولا تقرأ الطبقة أي ملف خارج المجلدات التي سجلها المطور، وتتحقق من المسار الحقيقي أولاً حتى لا يقودها اختصار إلى خارج هذه الحدود. ولكل عملية خارجية مهلة، وGit لا ينتظر كلمة سر أبداً، فلا يعلق الفحص على نافذة لا يراها أحد. أما فتح أدوات المطور فهو الفعل الوحيد المسموح، والأداة تُفتح منفصلة عن التطبيق.

8.2 أمان الخادم المحلي

الخادم برنامج يستقبل طلبات على الجهاز، فيجب ألا يصل إليه غير الواجهة. لذلك لا يقبل إلا طلبات الجهاز نفسه، ومعها رمز سري جديد في كل تشغيل لا تعرفه إلا الواجهة. ويرفض أي طلب يأتي من صفحة ويب، حتى لو كانت مفتوحة على الجهاز نفسه. ولا توجد في نسخة الإطلاق لوحة إدارة ولا صفحات لتصفح الواجهة البرمجية.

هذه الشروط كلها في مكان واحد قابل للاستبدال، فتحل محلها مصادقة حقيقية عند النشر على الإنترنت. والخادم لا يثق بالواجهة: **كل قاعدة على البيانات تُفرض في الخادم**، والواجهة تُخفي الزر الذي سيُرفض تسهيلاً للمطور فقط.

8.3 الأسرار

لا يطلب التطبيق كلمة سر ولا يحفظها. من ملف المتغيرات يقرأ الأسماء فقط، ولا يقرأ القيمة إلا لقائمة صغيرة من المتغيرات المعروفة بأنها غير حساسة، مثل رقم المنفذ. وكل ما عداها ممنوع افتراضياً، ولا يُستخرج جزء من قيمة سرية. وهذه القائمة مركزية، لا يستطيع مكتشف تقنية أن يوسّعها وحده.

ويحذف التطبيق اسم المستخدم وكلمة السر من رابط المستودع قبل أن يعرضه أو يحفظه. والسر المكتوب في كود المشروع يُعرض بالملف والسطر فقط، لا بقيمته. ومحتوى سجلات الأخطاء يُقرأ للعرض عند الطلب ولا يُحفظ أبداً، وإخفاء ما يشبه الأسرار عند عرضه طبقة حماية إضافية، لا الضمان الحقيقي.

8.4 الأخطاء والرسائل

لا يرجع الخادم جملاً، بل رموزاً ثابتة مع قيمها. لرد الخطأ شكل واحد في كل الطلبات: رمز الخطأ، والحقول التي سببته، وقيم إضافية إن لزم. ونتيجة كل فحص كذلك: رمز الفحص، وحالته، وقيم الرسالة، لا نص المشكلة. والواجهة تكتب الجملة من قالب لكل لغة، فتتغير صياغة رسالة دون أن يُلمس الخادم، وتظهر الرسالة نفسها بالعربية والإنجليزية.

والعطل لا يُخفى. الجزء الذي تعذر فحصه يظهر "لم يُفحص" مع سببه، ورسالة النظام الأصلية تُحفظ كما هي ليقرأها المطور. وحالة "الخادم لا يرد" تُعالج في نقطة الاتصال الوحيدة في الواجهة، لا في كل شاشة.

8.5 اللغتان

الواجهة بالعربية والإنجليزية، والمصطلحات التقنية بالإنجليزية في اللغتين، والاتجاه من اليمين إلى اليسار يأتي مع العربية. نصوص الواجهة كلها في قاموس واحد لكل لغة، ومعها قوالب رسائل الفحص وترجمة رموز الأخطاء. فكل رمز جديد يرجعه الخادم يحتاج سطراً في القاموسين، والمفتاح الناقص يُكتشف عند بناء التطبيق لا عند المستخدم.

ويفتح التطبيق مباشرة بلغة المطور ومظهره وحجم النص الذي اختاره، حتى قبل أن يجهز الخادم.

8.6 الذاكرة التلقائية

كل فحص يحفظ ما وجده لكل جزء لقطةً للحالة، لكن اللقطة الجديدة لا تُحفظ إلا إذا تغيّر شيء. يحسب التطبيق بصمة لمحتوى اللقطة بلا الأوقات، فإذا طابقت آخر لقطة، يُحدَّث وقت تأكيدها فقط. والجزء الذي تعذر فحصه لا ينشئ لقطة، فيبقى عمر آخر لقطة له صادقاً.

ويحتفظ التطبيق بكل اللقطات ثلاثين يوماً، ثم بلقطة واحدة لكل يوم حتى سنة. ولا يحذف أبداً اللقطة الحالية ولا آخر لقطة مختلفة عنها، فالمقارنة بحالة سابقة ممكنة دائماً مهما طال غياب المطور. وتحتفظ كل مشكلة بهوية ثابتة عبر الفحوصات وبتاريخ أول ظهورها، فلا يضيع هذا التاريخ حين تُحذف اللقطات القديمة.

وما يمكن حسابه لا يُحفظ. الاختلافات، وحالة كل قسم، وما تغيّر منذ زيارة المطور، ومركز الانتباه، كلها تُحسب عند بناء الشاشة من آخر اللقطات ومن معرفة المطور.

8.7 الفحص المتزامن

للفحص خيطا تنفيذ دائمان داخل الخادم: خيط للمشروع الذي يفتحه المطور، وخيط في الخلفية لفحص كل المشاريع ولأي فحص يحتاج التحديث من المستودع البعيد. فلا ينتظر المطور تحديثاً بطيئاً ليرى مشروعه. والمشروع الواحد لا يُفحص في الخيطين معاً.

جمع الحالة من الجهاز لا يلمس قاعدة البيانات، ويجري لأجزاء المشروع بالتوازي، ولكل جزء مهلة. أما الحفظ فيمر كله بكاتب واحد وبالدور، في عمليات قصيرة، وكل جزء يُحفظ فور انتهاء فحصه. فإذا انقطع الفحص بعده، بقي الجزء صحيحاً كاملاً. والجزء الذي تجاوز مهلته يُسجَّل "لم يُفحص" مع السبب، ويكمل الفحص الباقي. والخيط الذي علق في الجمع يُترك حتى ينتهي وحده، وهذا آمن لأنه لا يكتب شيئاً.

وسؤال "هل يجري فحص لهذا المشروع؟" يُجاب من قاعدة البيانات، لا من ذاكرة الخيط، فيبقى الجواب صحيحاً إذا تعدد من ينفّذ الفحص لاحقاً.

8.8 تحديث الشاشات

لا يوجد اتصال مفتوح بين الواجهة والخادم. ما دام هناك فحص جارٍ، تسأل الواجهة عن حالته كل ثانيتين تقريباً، ثم تتوقف وحدها. وهذه المتابعة في مكان واحد في الواجهة، لا في الشاشات.

وعند انتهاء فحص أي مشروع، وبعد أي كتابة ناجحة، تعيد الواجهة قراءة الشريط الجانبي، والشاشة المفتوحة إن كانت تعرض ذلك المشروع أو كانت عامة. والشاشة غير المفتوحة تُقرأ لحظة فتحها، فهي جديدة دائماً. وأثناء إعادة القراءة تبقى البيانات القديمة ظاهرة، ثم تُقارن بالجديدة فيُبرز ما تغيّر. والحد المعروف أن التحديث يصل خلال ثانيتين من انتهاء الفحص، لا في اللحظة نفسها.

8.9 سجل النشاط

يحتفظ التطبيق بسجل لأهم ما فعله ولقرارات المطور، مثل حسم اختلاف أو تحديث ملاحظة "أين توقفت". قرارات المطور تبقى فيه دائماً، وأحداث التطبيق سنة. والسجل دفتر يوميات يقرؤه المطور، لا مصدر للبيانات، فالبيانات الحقيقية في جداولها.

ولا يحتوي السجل أي نص من سجلات أخطاء المشاريع، ولا يُسجَّل فيه الفحص الناجح لمشروع واحد، حتى لا يمتلئ بما لا يفيد. وكتابة سطر فيه هي الشيء الوحيد الذي تطلبه كل الوحدات من مكان مشترك.

8.10 الاختبار

صُممت الطبقات لتُختبر كل منها وحدها. الحسابات الخالصة، مثل تقييم الفحوصات وبناء بيانات الشاشات، تُختبر بلا قاعدة بيانات. وخدمات كل وحدة تُختبر مع قاعدة بيانات للاختبار ونسخة وهمية من الطبقة المحلية، فلا يحتاج الاختبار مشاريع حقيقية ولا Windows. والأفعال المشتركة تُختبر للتأكد أن الفشل في منتصفها لا يترك شيئاً محفوظاً.

وفي الواجهة يُختبر كل مدير شاشة مع وسيط بيانات وهمي، والشاشات المهمة تُرسم في الاختبار باللغتين. وبعد أول نسخة تعمل، تُضاف اختبارات قليلة شاملة على الخادم الحقيقي.

9. القرارات المعمارية

كل قرار هنا أجاب عن سؤال واجهناه أثناء التصميم. مكتوب بنتيجته وسببه، ومعه البديل الذي رُفض، حتى لا يُعاد النقاش نفسه بلا سبب جديد.

#	القرار	السبب	ما رُفض ولماذا
1	شخصي أولاً: مستخدم واحد بلا حسابات ولا أدوار، وتصميم البيانات لا يفترض أن المستخدم مالكها الوحيد إلى الأبد.	يركز العمل على المشكلة الحقيقية، وميزات الفريق لا يمكن التحقق منها بلا فريق.	دعم الفريق من البداية: تعقيد كبير بلا مستخدمين يثبتون الحاجة. وتجاهل الفريق تماماً: إعادة بناء عند إضافته.
2	محلي أولاً: الخادم على جهاز المستخدم، وكل ما يلمس الجهاز في طبقة مستقلة تصبح وكيلاً محلياً عند النشر.	قراءة Git والمجلدات ممكنة من اليوم الأول بأبسط طريقة، وباب النشر يبقى مفتوحاً.	تطبيق مستضاف مع وكيل محلي من البداية: قطعتان قبل إثبات قيمة المنتج. ومحلي بلا عزل: النشر يصبح إعادة بناء.
3	اكتشاف محلي للقراءة فقط: بقراءة الملفات وأوامر قراءة ثابتة، بلا تشغيل لكود المشاريع ولا تغيير ولا كلمات سر.	ما يكتبه المطور مصدر القيمة المتوقعة، والاكتشاف مصدر القيمة الفعلية، فلا يكتب المطور ما يمكن اكتشافه.	المعرفة اليدوية وحدها: تلغي قيمة حالة Git، وتزيد الكتابة على المطور.
4	Windows أولاً، ونظام التشغيل معزول: كل ما يخص Windows خلف واجهة واحدة (Adapter)، لها نسخة وهمية للاختبار.	دعم نظام آخر يعني تنفيذاً جديداً للواجهة نفسها، والطبقة المحلية تُختبر على أي جهاز.	شروط على نظام التشغيل في كل مكان: تنشر كود Windows في النظام كله. ومكتبة متعددة الأنظمة بلا واجهة: لا تغطي كل ما نحتاجه، وأنواعها تنتشر في الكود.
5	نواة مستقلة عن التقنية، وDjango أولاً: الفحوصات العامة منفصلة، وما يخص كل تقنية في مكتشف بالواجهة نفسها (Strategy).	يخدم المستخدم الأول من اليوم الأول، ودعم تقنية جديدة إضافة مكتشف.	أداة Django فقط: تغلق الباب أمام غيرها. ونظام إضافات لعدة تقنيات من البداية: عمل كبير مبني على تخمين.
6	الفحص عند الطلب وفي الخلفية: فتح المشروع يفحصه وحده، وفحص كل المشاريع بطلب المطور، والشاشة لا تنتظر.	لا ينتظر المطور فحصاً ليرى مشروعه، ولا يُفحص ما لا يحتاجه.	فحص كل المشاريع عند فتح التطبيق: مؤجل إلى ما بعد الإصدار الأول.
7	ذاكرة تلقائية بسياسة حذف متدرجة: لقطة عند التغيّر فقط، وكل اللقطات ثلاثين يوماً ثم واحدة يومياً حتى سنة، واللقطتان الأخيرتان لا تُحذفان.	تاريخ مفيد لسنة بحجم محدود، والمقارنة بحالة سابقة ممكنة دائماً.	الحذف بالعمر وحده: يفقد الحالة السابقة لمشروع طال غيابه. وعدد ثابت من اللقطات: يفقد تاريخ المشاريع النشطة بسرعة. وعدم الحذف أبداً: حجم بلا سقف.
8	ملف المتغيرات: أسماء فقط، وقيم قليلة غير حساسة من قائمة سماح مركزية.	معلومات مفيدة مثل المنفذ ووضع التطوير دون أي سر، بقواعد بسيطة يصعب الخطأ فيها.	أسماء فقط دائماً: تضيع معلومات آمنة. وأن يقرر المطور لكل مشروع: يضع قرار الأمان عليه في كل مرة.
9	الوثائق روابط، ويُرفع المهم فقط، ولا تُحفظ بيانات دخول.	يحل ضياع وثائق العملاء دون أن يصبح التطبيق منصة تخزين.	روابط فقط: لا تحل ضياع الوثائق. وتخزين كل الملفات: يكرر الملفات ويخرج التطبيق عن هدفه.
10	تطبيق واحد مدمج: الواجهة تشغّل الخادم وتغلقه، وSQLite، ومنفذ حر، ورمز سري جديد في كل تشغيل، ومتابعة دورية بدل اتصال مفتوح، والفحص في خيط داخل الخادم.	تثبيت بلا أي خدمة، وسلوك واحد في التطوير والإطلاق، ولا تعارض مع منافذ مشاريع المطور.	PostgreSQL: يحتاج كل مستخدم تثبيت خادم. واستضافة الخادم: لا يرى الجهاز. ومنفذ ثابت: قد يتعارض مع مشاريع المطور. واتصال مفتوح: يعقّد التغليف بلا فائدة لفحص يستغرق ثوانيَ.
11	خيطان للفحص وكاتب واحد: خيط للمشروع المفتوح وخيط للخلفية، والحفظ كله بالدور.	لا ينتظر المشروع المفتوح تحديثاً بطيئاً، ولا تتزاحم الكتابة.	خيط واحد بطابور: التحديث من المستودع البعيد في فحص كل المشاريع قد يؤخر المشروع المفتوح عشرات الثواني.

#	القرار	السبب	ما رُفض ولماذا
12	نسخة احتياطية مشفّرة تدمج ولا تحذف: ما كتبه المطور وملفاته المرفوعة في ملف واحد مشفّر بكلمة سر ومحمي من التلاعب، والاستيراد يضيف الناقص ويغلب فيه الأحدث.	ملف صغير مستقل عن بنية الجداول، والاستيراد لا يخسر شيئاً على الجهاز.	نسخ ملف قاعدة البيانات: ينسخ ما يُعاد اكتشافه، ويرتبط ببنية الجداول. وتشفير ملفات zip: ضعيف. والاستبدال عند الاستيراد: يمسح ما على الجهاز.
13	عميل وخادم، وخادم مقسوم إلى وحدات: وبعد النشر وكيل محلي مع خادم على الإنترنت.	برنامج واحد لفريق صغير، والواجهة لا تُكتب مرتين عند النشر.	تطبيق بلا خادم: يعيد كتابة المنطق عند النشر. وخدمات صغيرة منفصلة: تعقيد توزيع بلا فائدة لمستخدم واحد. والأحداث والطوابير الآن: تعقيد زائد.
14	طبقة خدمات وطبقة أفعال مشتركة في الخادم.	للمنطق نفسه أكثر من مدخل: طلبات الواجهة، والفحص، والاستيراد، وبدء الخادم. والأفعال تكتب في أكثر من وحدة.	المنطق في استقبال الطلبات: لا يُعاد استعماله ولا يُختبر وحده. والمنطق في الجداول: ملفات ضخمة لأفعال تمس عدة وحدات. وArchitecture Clean كاملة: كتابة كل شيء مرتين. والأحداث المخفية: تسلسل لا يُرى واختبار صعب.
15	MVVM مع Riverpod في الواجهة، ونقطة اتصال واحدة، ومجلد لكل شاشة باسمها.	الطريقة التي يوصي بها دليل Flutter، وإعادة القراءة بعد الفحص سهلة، والنسخ الوهمية تُركّب بسطر.	كل شيء في الشاشة: المنطق مربوط بالرسم. وBloc: ملفات كثيرة لكل شاشة. وGetX: سلوك مخفي. وتوليد كود الاتصال الآن: ممكن لاحقاً خلف وسيط البيانات.

10. الجودة

أهداف الجودة في القسم 1.2 كلمات عامة حتى تُختبر بموقف حقيقي. يربط هذا القسم كل هدف بمتطلباته، ثم يضع مواقف ملموسة يمكن أن يُسأل عنها التطبيق.

10.1 أهداف الجودة ومتطلباتها

#	الهدف	المتطلبات في وثيقة المتطلبات
1	الأمان	SAF-01..04, SEC-01..07
2	الصدق	BR-01..07, UI-03
3	السرعة الظاهرة	PRF-01..03, UI-02, SCN-09, SCN-10
4	الموثوقية	REL-01..04, SCN-04..06, SCN-12
5	قابلية التطوير	MNT-01..04

10.2 سيناريوهات الجودة

#	الموقف	ما يحدث	ما يضمنه في التصميم
1	يفتح المطور مشروعاً لم يُفحص منذ شهر.	تظهر الشاشة فوراً بآخر حالة معروفة، وعمر كل معلومة واضح، والفحص يجري في الخلفية.	طلب البيانات منفصل عن طلب الفحص، والشاشة تُبنى من آخر اللقطات.
2	Docker متوقف أثناء الفحص.	تظهر حالة الحاويات "غير معروفة"، لا "متوقفة"، وتظهر بقية النتائج طبيعياً.	كل مصدر يُقرأ وحده، وغياب المصدر نتيجة "لم يُفحص" مع السبب.
3	مجلد جزء على قرص بطيء يعلق أثناء القراءة.	بعد المهلة يظهر الجزء "لم يُفحص" مع السبب، ويكمل الفحص الباقي.	مهلة لكل جزء ولكل عملية خارجية، والجمع منفصل عن الحفظ.
4	صفحة ويب خبيثة مفتوحة في المتصفح تحاول الاتصال بالخادم.	يُرفض الطلب.	شروط الأمان في مكان واحد: طلبات الجهاز فقط، والرمز السري، ورفض أي طلب من صفحة ويب.
5	يغلق المطور التطبيق أثناء فحص.	يبقى ما اكتمل من الأجزاء، ويظهر أن فحص المشروع الأخير انقطع، بلا رسالة عطل.	الإغلاق يمر بالخادم قبل إيقافه، وكل جزء يُحفظ فور انتهاء فحصه.
6	يتوقف الخادم فجأة.	رسالة واحدة واضحة، وإعادة تشغيل، ولا فحص معلّق بعدها.	الواجهة تراقب الخادم، والخادم يعالج ما بقي من التشغيل السابق عند بدئه.
7	يُضاف دعم لتقنية مشاريع جديدة.	تُفحص الأجزاء المبنية بها في أول فحص بعد الإضافة.	مكتشف جديد بالواجهة نفسها، دون تغيير بقية الخادم.
8	يُقرر نشر التطبيق على الإنترنت.	يتغير عنوان الخادم وتُضاف شاشة دخول، والطبقة المحلية تصبح برنامجاً صغيراً على جهاز المستخدم.	العنوان والدخول في مكان واحد، والطبقة المحلية خلف واجهات مجردة، والقراءة من كل وحدة في مكان واحد.

11. المخاطر والدين التقني

لا يوجد تصميم بلا ثمن. هذا القسم يذكر بصراحة ما نعرف أنه ناقص، أو قد يسوء، أو قبلناه عن قصد، وما نفعله في كل حالة.

#	الخطر أو الدين	أثره	ما نفعله
1	أرقام الأداء غير محددة بعد: زمن ظهور الشاشة، ومهلة الجزء، وحجم قاعدة البيانات مع الوقت.	مهلة قصيرة تكثر "لم يُفحص"، ومهلة طويلة تؤخر الفحص.	تُقاس بعد أول نسخة تعمل. والأرقام إعدادات تتغير دون تغيير التصميم.
2	لا ذاكرة مؤقتة لبيانات الشاشات، فكل شاشة تُحسب من قاعدة البيانات في كل مرة.	قد تبطؤ الشاشات مع كثرة المشاريع.	تُقاس بعد التجربة، وتُضاف في الإصدار الأول نفسه إن ظهر بطء.
3	قاعدة البيانات تقبل كاتباً واحداً في اللحظة.	قد يتأخر الحفظ عند فحوصات كثيرة.	كاتب واحد بالدور في عمليات قصيرة. ولا يُغيَّر الاختيار إلا بقياس يثبت الحاجة.
4	قواعد الحدود بين وحدات الخادم، وبين الشاشات والخادم، لا تفرضها أداة.	قد يتسرب اعتماد دائري، أو طلب مكتوب داخل شاشة.	مراجعة الكود، وبحث بسيط عن أي رابط داخل مجلد الشاشات، ويمكن لاحقاً إضافة أداة تمنع الاعتماد الخاطئ.
5	الخيط الذي يعلق بعد المهلة يُترك ولا يُوقف.	يبقى يعمل في الخلفية حتى ينتهي أمره.	آمن لأنه في مرحلة الجمع ولا يكتب شيئاً، والجزء يظهر "لم يُفحص".
6	آلية إغلاق الخادم إذا توقفت الواجهة، ومنع تشغيل نسختين، تُحسم بتجربة عند كتابة الكود.	إذا فشلت، قد يبقى خادم يعمل وحده، أو يحاول خادمان العمل على قاعدة واحدة.	حماية على طبقتين: الواجهة تمنع النسخة الثانية، والخادم يحجز مجلد البيانات ويرفض البدء إذا كان محجوزاً.
7	مكتشف التقنية لـ Django وحده في الإصدار الأول.	المشاريع بتقنيات أخرى تحصل على الفحوصات العامة فقط، مع تنبيه واضح للجزء.	إضافة مكتشف جديد لا تغيّر بقية التطبيق.
8	Windows فقط.	لا يعمل التطبيق على نظام آخر.	كل ما يخص نظام التشغيل معزول في مكوّن واحد.
9	الخادم المغلّف ملف كبير نسبياً، وقد يطلب Windows تأكيداً عند تشغيل برنامج غير موقّع أول مرة.	انطباع أول أقل سلاسة.	مقبول في الإصدار الأول.
10	التحديث بعد الفحص يصل خلال ثانيتين، لا في اللحظة نفسها.	تأخير بسيط في ظهور النتيجة.	مقبول. الاتصال المفتوح يعقّد التغليف بلا فائدة لفحص يستغرق ثوانيَ.
11	الانتقال إلى الإنترنت: الحسابات أصعب ما فيه، فكل معلومة ستحتاج مالكاً، والوكيل المحلي سيجيب عن كل طلب يقرأ الجهاز.	عمل كبير عند قرار النشر.	القراءة من كل وحدة في مكان واحد، والطبقة المحلية خلف واجهات مجردة، والعنوان والدخول في مكان واحد في الواجهة.
12	كلمة سر النسخة الاحتياطية المنسية تعني خسارة النسخة.	لا طريقة للاستعادة.	معلن للمطور. وهو ثمن التشفير الإلزامي.
13	بعد الاستيراد على جهاز جديد يبدأ تاريخ المشاريع من جديد.	"منذ متى ظهرت المشكلة" يبدأ من أول فحص على الجهاز الجديد.	مقبول، فالنسخة تحفظ ما لا يمكن اكتشافه، والفحص يعيد الباقي.
14	الواجهة تعتمد على توليد كود عند البناء، وعلى مكتبة حالة تتغير بين إصداراتها الكبيرة.	خطوة بناء إضافية، واحتمال كسر عند الترقية.	إصدار مثبَّت للمكتبات، والترقية قرار مقصود.

12. المصطلحات

هذه المصطلحات تتكرر في الوثيقة. ومفاهيم المنتج كاملة، بخصائصها وعلاقاتها، في النموذج المفاهيمي.

المصطلح	المعنى
المشروع	وحدة عمل يحددها المطور، وتتكون من جزء أو أكثر.
الجزء	قسم من المشروع مرتبط بمستودع Git، مثل الواجهة الخلفية أو الأمامية.
المستودع	مستودع Git على الجهاز. وقد يضم عدة أجزاء، لكل منها مجلده.
الفحص	قراءة حالة المشروع من الجهاز في لحظة معينة.
لقطة الحالة	ما وجده الفحص لجزء واحد، مع وقت الفحص.
الذاكرة التلقائية	تاريخ لقطات الحالة عبر الوقت، يتكوّن دون أن يكتب المطور شيئاً. وهي أيضاً اسم المكوّن الوحيد الذي يحفظ هذا التاريخ.
القيمة المتوقعة والقيمة الفعلية	الأولى يكتبها المطور، والثانية يجدها الفحص.
الاختلاف	عدم تطابق القيمة الفعلية مع القيمة المتوقعة.
شاشة الاستئناف	الشاشة التي تظهر عند فتح مشروع.
الواجهة	تطبيق سطح المكتب الذي يراه المستخدم، مبني بـ Flutter.
الخادم	البرنامج الذي يحمل منطق التطبيق وبياناته، مبني بـ Django، ويعمل في الخلفية على الجهاز نفسه.
الواجهة البرمجية	الطلبات التي تتبادلها الواجهة والخادم، وعقدها موثق في عقد الواجهة البرمجية.
الحاوية	بمصطلحات C4: برنامج يعمل وحده أو مخزن بيانات. وليست حاوية Docker.
المكوّن	دور داخل الخادم بمصطلحات C4، مثل منسّق الفحص.
الوحدة	قسم من كود الخادم له جداوله وحدوده، والخادم تسع وحدات.
الطبقة	مجموعة أجزاء في المستوى نفسه، تعتمد على ما تحتها فقط.
الطبقة المحلية	الطريق الوحيد إلى الجهاز داخل الخادم، للقراءة وفتح الأدوات فقط.
الفعل المشترك	فعل واحد من المستخدم يكتب في أكثر من وحدة، مثل إزالة مشروع له عميل.
وحدة الشاشات	وحدة في الخادم تجمع ما تعرضه كل شاشة، بالقراءة والحساب فقط.
مدير الشاشة	الجزء في الواجهة الذي يحمل حالة الشاشة وأفعالها.
وسيط البيانات	الجزء في الواجهة الذي يرسل طلبات وحدة واحدة من الخادم. ويقابل نمط Repository، ولا علاقة له بمستودع Git.
نقطة الاتصال الوحيدة	المكان الوحيد في الواجهة الذي يعرف عنوان الخادم والرمز السري.
الرمز السري	قيمة عشوائية جديدة في كل تشغيل، يرفض الخادم أي طلب بدونها.

المصطلح	المعنى
خيط التنفيذ	مسار عمل داخل الخادم يعمل بالتوازي مع غيره، فلا يوقف الفحص باقي التطبيق.
المهلة	أقصى وقت يُنتظر فيه جزء أو عملية خارجية، وبعده يُسجَّل "لم يُفحص" مع السبب.
المتابعة الدورية	سؤال الواجهة عن حالة الفحص كل ثانيتين تقريباً ما دام جارياً، بدل اتصال مفتوح.
انقطاع الفحص	فحص لم يكتمل لأن التطبيق أُغلق أو توقف. يبقى ما اكتمل من أجزائه، ولا يُعتبر المشروع مفحوصاً كاملاً.
التطبيق المدمج	الواجهة والخادم في برنامج تثبيت واحد، والواجهة تشغّل الخادم وتغلقه.
مجلد البيانات	المجلد الوحيد الذي يكتب فيه التطبيق: قاعدة البيانات، والملفات المرفوعة، والملفات المؤقتة.
الوكيل المحلي	برنامج صغير على جهاز المستخدم يحل محل الطبقة المحلية إذا نُشر الخادم على الإنترنت.